

Relatório 17 – Redes Neurais Convolucionais 2

Edryck Freitas Nascimento

1. Introdução

O objetivo desta aula foi aprender sobre redes neurais convolucionais e a prática de construção de uma CNN. A tarefa consistia em assistir as seções 5 a 11 do curso *Deep Learning: Convolutional Neural Networks in Python* e realizar um exemplo prático demonstrando a compreensão do conteúdo visto.

2. Desenvolvimento

- **Obtenção de Dados:** Inicialmente busquei o *dataset* no catálogo do Tensorflow, encontrei o Scicite que me chamou atenção.
- **Preparação dos Dados:** Os dados brutos são processados. Esta etapa incluiu limpeza, como ao carregar o *dataset* vinham só as colunas de rótulos e a de dados, não foi preciso uma limpeza e divisão dos conjuntos (treino, validação e teste). Nesta etapa, não foi necessário a divisão dos conjuntos, pois, já vinham separados.
- **Configuração do Modelo:** A arquitetura utilizada foi:
 - Uma camada de *Embedding* para representar as palavras como vetores densos de dimensão 20.
 - Em seguida, três blocos convolucionais com filtros de tamanho crescente (32, 64 e 128) intercalados com *MaxPooling1D* para redução dimensional progressiva, no último bloco, um *GlobalMaxPooling1D* agrega as *features* mais relevantes de toda a sequência em um vetor de 128 dimensões.
 - A classificação final é feita por uma camada *Dense* de 64 neurônios com ReLU, seguida de *Dropout* de 0.5 e uma camada de saída com 3 neurônios e ativação *Softmax* (uma por classe de intenção de citação (*background, method, result*)).
 - A função de perda escolhida foi a *Sparse Categorical Cross-Entropy* para classificação multiclasse com *labels* inteiros e o otimizador Adam com taxa de aprendizado de 0.0001, porque foi o valor que demonstrou uma convergência estável em comparação com a taxa padrão de 0.001 que causava oscilações na validação.
- **Treino:** O modelo foi treinado com *batch size* de 16 e taxa de aprendizado de 0.0001 por 15 épocas. Escolhi o batch size menor porque

com 32 (que é o padrão da aula) o modelo entrava em *overfitting* já na primeira época, com acurácia de treino em 99% e validação entre 73% e 78% sem melhorar. Quando reduzi para 16, o treinamento ficou mais estável e o modelo começou a aprender de forma gradual, saindo de 57% na primeira época.

- **Validação:** O melhor resultado foi na época 4, com `val_acc` de 69.65% e `val_loss` de 0.7519. A partir daí, a acurácia de validação estabilizou entre 69% e 70% enquanto a *loss* continuou subindo até 1.79 na época 15, mostrando que o modelo passou do ponto ideal e entrou em *overfitting*. No conjunto de teste, a acurácia ficou em 65.41%, o que é um pouco baixo na validação já que o modelo nunca viu esses dados durante o treinamento.

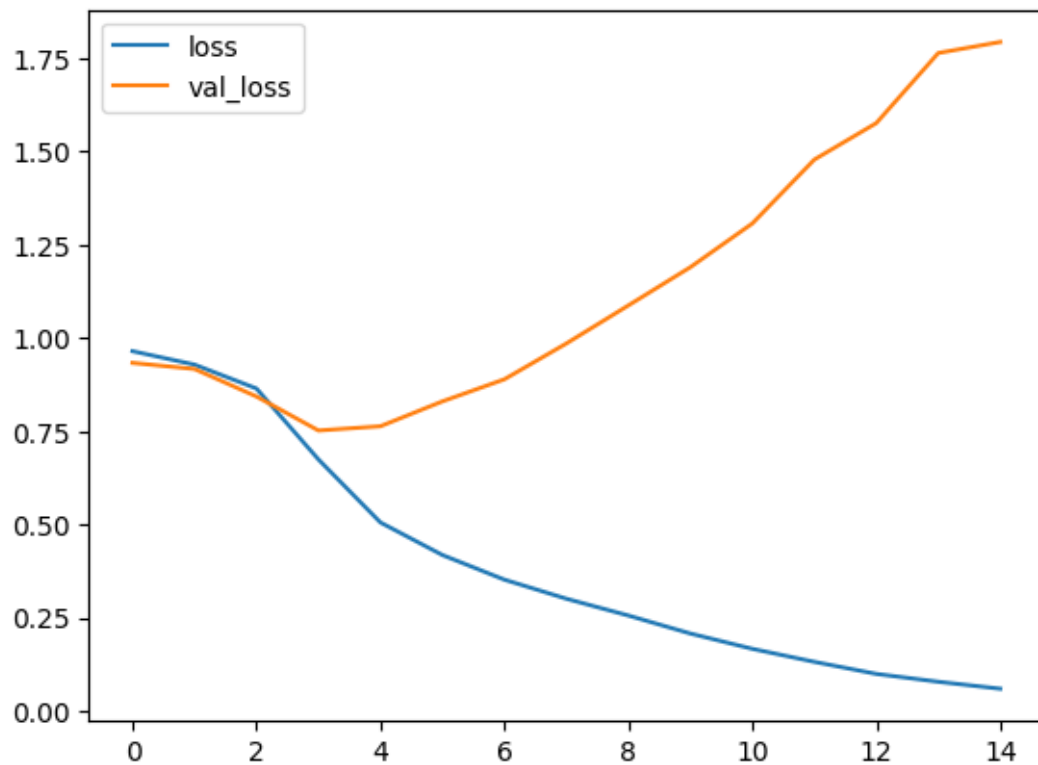


Figura 1: Gráfico da perda e perda na validação obtido após treino do modelo. (código da aula adaptado ao meu cenário)

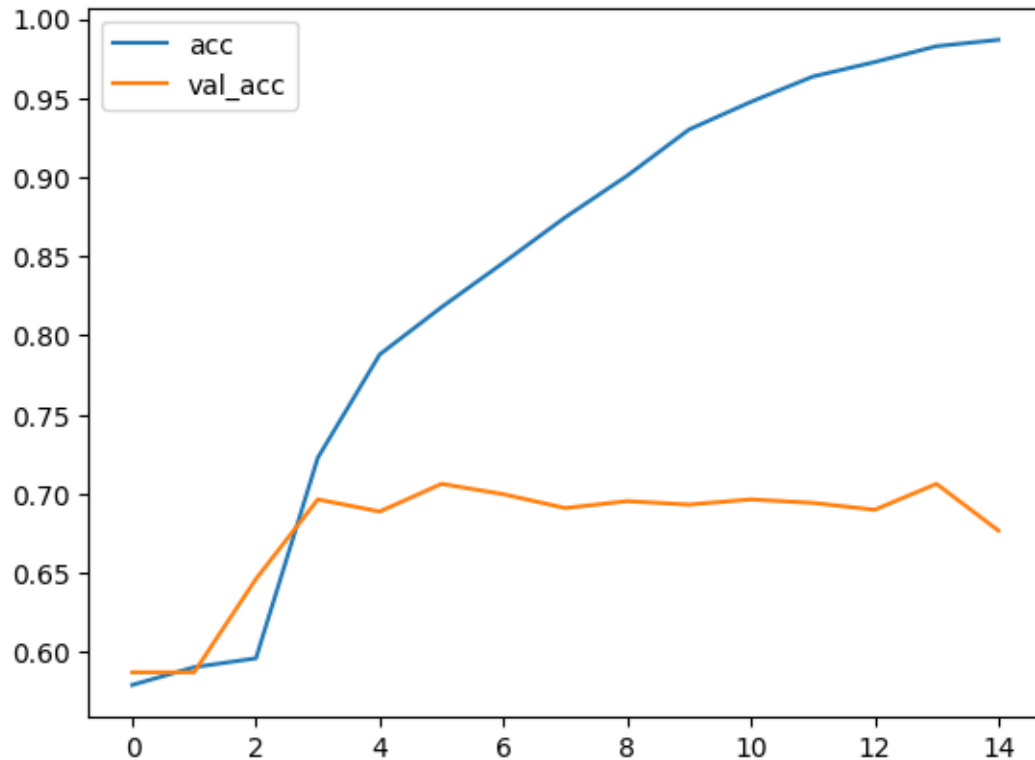


Figura 2: Gráfico de comparação da aurácia durante treino e validação obtido após treino do modelo. (código da aula adaptado ao meu cenário)

3. Conclusão

A atividade mostrou na prática como redes neurais convolucionais conseguem ser aplicadas em texto, tratando sequências de palavras do mesmo jeito que tratam padrões em imagens. Usar o scicite para classificar a intenção de citações científicas foi um bom teste, já que é um problema real com três classes distintas e um volume de dados moderado, embora talvez precisasse de mais. O maior desafio foi o *overfitting* que o modelo decorava os dados de treino muito rápido e assim foi necessário ajustar algumas vezes para conseguir uma convergência mais estável. O resultado final de 65% no conjunto de teste mostra que o modelo aprendeu alguma coisa, só que ainda tem limitações o que provavelmente é por causa do desbalanceamento entre as classes e da dimensão pequena do *embedding*.

Referências

Curso:

Lazy Programmer Team. <https://www.udemy.com/course/deep-learning-convolutional-neural-networks-theano-tensorflow/>

Dataset:

TensorFlow. <https://www.tensorflow.org/datasets/catalog/scicite?hl=pt-br>